

人工智能创新赛 项目研究报告

abb-offline-coder

面向 ABB 喷涂机器人的 离线 AI 编程助手

*Retrieval-Augmented Offline Code Generation
for Industrial Painting Robots*

比赛赛项 人工智能创新赛

队伍名称 人工智能创新 001

项目类别 工业大模型 · 检索增强生成 (RAG)

技术形态 完全离线 · 命令行 AI 编程助手

“把中文一句话变成 ABB RAPID 喷涂程序”

在线演示 hollis36.github.io/abb-offline-coder • 源码 github.com/Hollis36/abb-offline-coder

报告日期: 2026 年 6 月 | 版本 v0.2 | MIT 开源

摘要

工业机器人离线编程长期面临**专业门槛高、厂商语言封闭、现场数据不能外传**三重困境。ABB 喷涂机器人采用专用的 RAPID 语言与 IRC5/IRC5P 控制器体系，编写一段合格的喷涂程序需要工程师同时掌握运动指令、喷涂工艺（笔刷参数、喷枪时序）、坐标标定与控制器 IO 配置，培养周期以月计；而喷涂车间多为**物理隔离的无网环境**，主流云端 AI 编程工具完全无法落地。

本项目 **abb-offline-coder** 提出并实现了一套**完全离线、可在工业 PC（CPU、无 GPU）上运行的领域专精 AI 编程助手**：用户用中文自然语言描述喷涂需求，系统经“查询改写 → 向量 + BM25 混合检索 → 上下文装配 → 本地大模型推理 → RAPID 静态校验与后处理”六级流水线，直接产出符合 ABB 规范、可导入 RobotStudio 的 .mod 程序。

核心创新包括：（1）以本地 Qwen2.5-Coder 量化模型 + 检索增强（RAG）注入 ABB 官方手册知识，使小模型也能正确引用 MToolTCPCalib、TriggIO 等专业指令；（2）面向 **IRC5 / IRC5P 双控制器**的自适应代码风格切换（MoveL+SetDO 对 PaintL/PaintC+brushdata）；（3）**生成—校验闭环**——内置 11 类 RAPID 轻量静态校验（块配对、大小写、分号、笔刷参数、默认 TCP、IO 白名单等）并赋予可追溯的错误码；（4）**Pack&Go 一键交付**——从一句话需求直接生成包含 .mod + BASE.sys + T_ROB1.pgf + README 的可加载控制器目录；（5）**一键离线迁移**——把代码、模型、向量库、手册打成单文件在隔离设备上还原。

工程上，项目实现约 4500 行 Python（37 个源文件），配套 22 个测试文件、约 149 个测试用例，以 10 本真实 ABB 官方手册构建了 7497 片段的向量知识库，端到端单次生成在普通工业 PC 上约 20–35 秒。本报告系统阐述其选题意义、总体架构、关键技术、创新点、工程质量与应用价值，并讨论安全合规与未来演进路线。

关键词：工业大模型；检索增强生成（RAG）；离线部署；ABB RAPID；喷涂机器人；代码生成；本地大语言模型

目录

摘要	1
1 项目背景与选题意义	4
1.1 行业痛点：喷涂机器人编程的“三高一隔离”	4
1.2 机遇：本地大模型 + 检索增强	4
1.3 选题意义	4
2 项目概览与开源信息	5
2.1 项目定位（公开发布）	5
2.2 项目特性一览（来源：在线演示页 9 张特性卡片）	5
3 相关工作与对比	7
4 系统总体设计	7
4.1 设计原则	7
4.2 分层架构	7
4.3 核心数据流：单次生成流水线	8
4.4 技术栈	8
5 关键技术与实现	10
5.1 中文查询改写：低成本提升检索命中	10
5.2 向量 + BM25 混合检索	10
5.3 上下文装配：来源标注 + 预算控制	11
5.4 本地 LLM 推理与提示工程	11
5.5 RAPID 后处理：生成—校验闭环	11
5.6 双控制器模式：IRC5 与 IRC5P 自适应	12
5.7 Pack&Go 一键交付	13
5.8 一键离线迁移	13
6 创新点总结	14
7 工程质量与实测	14
7.1 代码规模与测试	14
7.2 知识库与模型规格	14
7.3 性能基准（工业 PC：i5 + 16 GB RAM，无 GPU）	14

8 应用案例：车门外板 Z 字喷涂	16
8.1 IRC5 模式输出 (MoveL + SetDO)	16
8.2 IRC5P 模式输出 (PaintL + brushdata)	17
9 安全、合规与伦理	17
10 局限性与未来工作	18
10.1 当前局限	18
10.2 演进路线	18
10.3 方法论可迁移性	18
11 结论	18
参考资料与项目信息	19

1 项目背景与选题意义

1.1 行业痛点：喷涂机器人编程的“三高一隔离”

喷涂是汽车、家电、3C、家具等行业的关键工序，ABB 是全球喷涂机器人主要供应商之一（IRB 52 / 5400 / 5500 / 5710 等机型）。其编程语言 **RAPID** 是 ABB 专有的、大小写敏感的结构化语言，喷涂场景还要叠加专用工艺。一名工程师要独立写出可上机的程序，需同时掌握以下相互交织的知识：

- **运动与轨迹**：MoveJ/MoveL/MoveC、转弯区 zonedata、速度 speeddata、Z 字/弓字扫描的行距与重叠规划；
- **喷涂工艺**：笔刷数据 brushdata（流量、扇幅、雾化压力）、喷枪开关的提前/延迟时序、TriggIO 距离触发同步；
- **标定与坐标**：工具坐标 tooldata 的 TCP 4 点法标定、工件坐标 wobjdata；
- **控制器配置**：IO 信号必须在控制器 EIO.cfg 中注册，否则现场报 40212 “信号未定义”；IRC5P 还依赖 RobotWare Paint 选项。

由此形成“三高”——学习门槛高、试错成本高、对人依赖高。更棘手的是“一隔离”：出于工艺保密与功能安全，喷涂产线的控制器与编程电脑通常**物理隔离、完全无网**。这意味着 GitHub Copilot、ChatGPT、Cursor 等依赖云端的 AI 编程工具在喷涂现场根本无法使用，现场数据也不允许上传外部服务。

1.2 机遇：本地大模型 + 检索增强

近年来，代码专精的开源大模型（如 Qwen2.5-Coder 系列）在 7B 量级即可达到可用的代码生成质量，配合 4-bit 量化后仅需约 4.5 GB 内存即可在纯 CPU 上推理；检索增强生成（RAG）则能把厂商手册等私有领域知识动态注入提示词，显著缓解小模型“知识不足/幻觉”问题。两者结合，使“完全离线的工业领域代码助手”第一次具备工程可行性——这正是本项目的技术机遇窗口。

1.3 选题意义

1. **解决真问题**：直面喷涂车间“无网 + 高门槛”的真实落地障碍，而非又一个云端 demo；
2. **数据安全**：需求描述、工艺参数、历史程序全程不出厂，天然满足工业信息安全合规；
3. **降本增效**：把“写一段喷涂程序”从数小时的专家工作，压缩为数十秒的自然语言交互 + 人工复核；

4. **方法论可迁移**：本项目的“离线 RAG + 领域校验闭环”范式可推广到其它工业机器人语言（KUKA KRL、FANUC TP、库卡/发那科）与 PLC 等封闭生态。

2 项目概览与开源信息

2.1 项目定位（公开发布）

项目已在 GitHub 开源并部署在线演示页，对外的一句话定位为：

“把中文一句话变成 *ABB RAPID* 喷涂程序——本地小模型 + 官方手册 RAG，工业 PC 断网可用。”

（仓库简介：离线 AI 编程助手·中文需求 → ABB RAPID 喷涂程序·本地 LLM（Qwen2.5-Coder）+ 官方手册 RAG ·工业 PC 断网可用。）

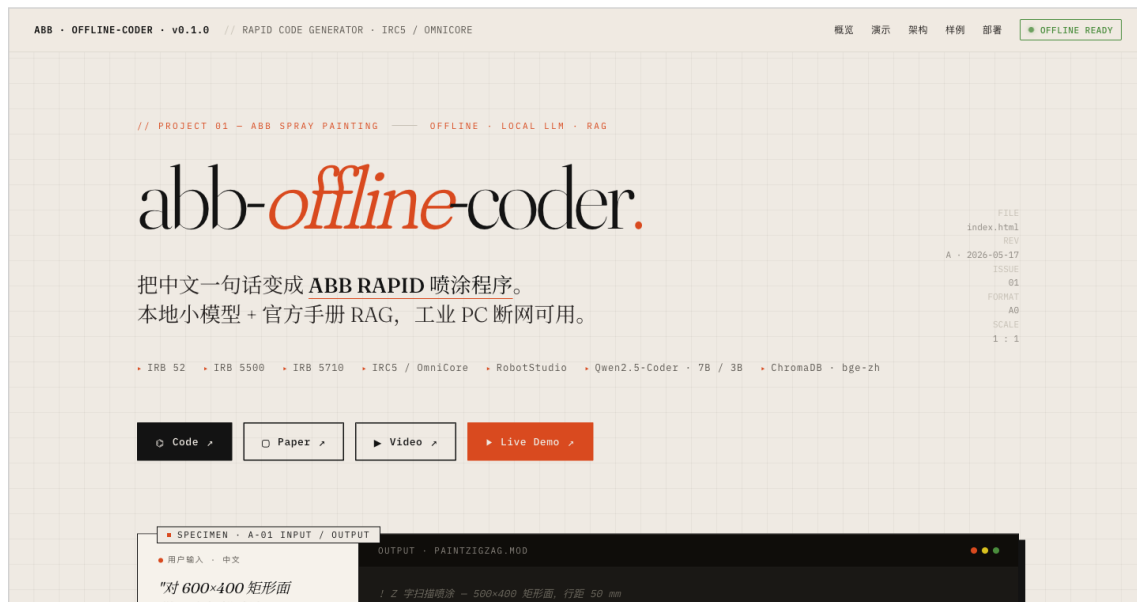


图 1. 项目在线演示页横幅（部署于 GitHub Pages，离线可浏览）。

2.2 项目特性一览（来源：在线演示页 9 张特性卡片）

在线演示页以 9 张特性卡片（F-01 ~ F-09）概括项目能力，原文摘录如下表，与本报告后续章节的技术实现一一对应。

说明：演示页为项目早期快照（标注约 3664 行 / 67 测试）；当前仓库 HEAD 经实测已增长至约 4506 行 / 156 个通过用例（见第 7 节），体现项目持续演进。

表 1. 项目开源信息（来源：GitHub 仓库与在线演示页）

项	值
在线演示页	https://hollis36.github.io/abb-offline-coder/ （GitHub Pages，离线可浏览）
开源仓库	https://github.com/Hollis36/abb-offline-coder
开源许可	MIT License
主要语言	Python（占比约 87.3%）
项目徽章	Live Demo · Online · License: MIT · Python 3.10+ · tests passed
技术标签	rag · ollama · qwen · chromadb · offline-llm · abb-robotics · rapid-language · spray-painting · robotstudio · industrial-robot · chinese-nlp · cli-tool

表 2. 项目核心特性一览（演示页 F-01~F-09，原文摘录）

编号	特性	演示页原文描述
F-01	真·离线	LLM 推理、嵌入模型、向量检索全部本地
F-02	中文支持	中文需求 → 中文注释 + 标准英文 RAPID 代码
F-03	官方手册 RAG	从 10 本 ABB PDF 切出 7497 片，向量 + BM25 混合检索
F-04	喷涂专精	few-shot 覆盖直线扫描 / Z 字扫描 / TriggIO 同步 / TCP 校准 / 圆弧轨迹
F-05	轻量·工业友好	Q4 量化模型 4.5 GB，CPU 推理 ≈ 25s
F-06	语法自校验	正则校验常见错误类别（本报告已扩展至 11 类 + 错误码）
F-07	跨平台	Windows / macOS / Linux，UTF-8 与 CRLF 自适应
F-08	一键离线打包	有网工作站打包 → tar.gz / zip → 拷贝 → 工业 PC
F-09	测试覆盖	核心逻辑 90–100% 覆盖（本报告实测已增长至 156 用例）

3 相关工作与对比

表 3. abb-offline-coder 与现有方案的定位对比

方案	离线	领域专精	生成即校验	目标场景
GitHub Copilot / Cursor	✗	通用	✗	联网的通用软件开发
ChatGPT / 通用云端 LLM	✗	通用	✗	联网问答，需人工搬运代码
RobotStudio 图形示教	✓	ABB 专精	部分	手工点选，无自然语言
通用本地 LLM（裸 Ollama）	✓	通用	✗	缺手册知识，易幻觉
abb-offline-coder（本项目）	✓	喷涂专精	✓	无网工业 PC，中文一句话生成可上机程序

与上述方案相比，本项目的差异化定位是同时满足“**离线 + 领域专精 + 生成即校验 + 自然语言交互**”四项，填补了“无网工业现场的中文 AI 编程”空白。

4 系统总体设计

4.1 设计原则

项目遵循六条工程原则，确保在资源受限的工业 PC 上稳定、可演进：

1. **离线优先**：所有运行时依赖（LLM、嵌入模型、向量库）必须断网可用；
2. **资源敏感**：默认 CPU 推理、无需独立 GPU，量化模型 4.5 GB 内存可跑；
3. **可演进**：知识库可持续增量扩充，模型可随硬件升级替换，配置零代码改动；
4. **小文件、单一职责**：每模块 200–400 行，便于审查与测试；
5. **不可变数据**：核心数据模型全部 frozen，副作用集中在 Ollama / ChromaDB 边界；
6. **可测试**：以纯函数为主，逻辑层覆盖率高。

4.2 分层架构

系统采用清晰的四层架构：CLI 层负责交互，编排层（Agent）串联流水线，能力层分为 RAG 检索 / 本地 LLM / RAPID 后处理三大子系统，底层是离线知识库构建。

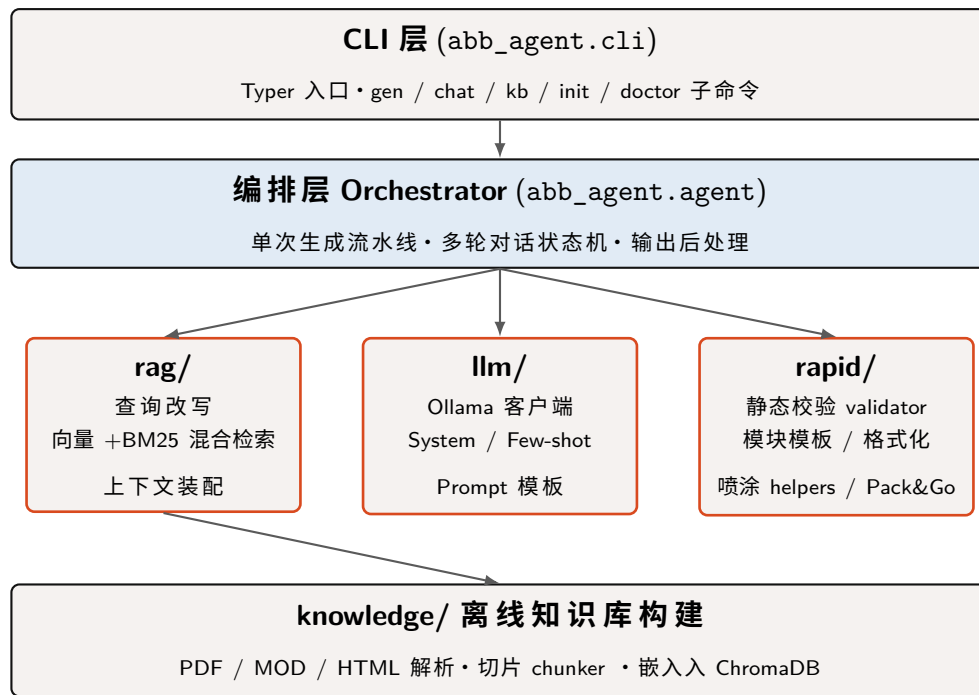


图 2. abb-offline-coder 四层架构 (CLI / 编排 / 能力 / 知识库)

4.3 核心数据流：单次生成流水线

一次“中文需求 → .mod 文件”的处理是一条函数式纯流水线，每一步的产物都是不可变 dataclass，便于测试与复用：

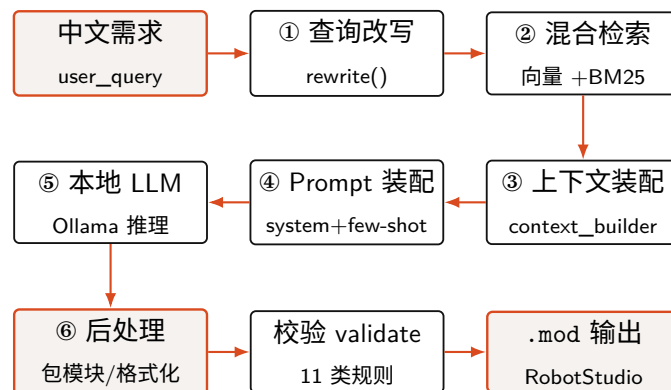


图 3. 单次生成的六级流水线 (query rewrite → retrieve → assemble → prompt → LLM → post-process)

上述流水线在项目在线演示页上有可交互的实现：用户改一行需求即可看到各阶段状态流转与 .mod 即时重生成，便于直观理解“中文需求 → 可上机程序”的全过程。

4.4 技术栈

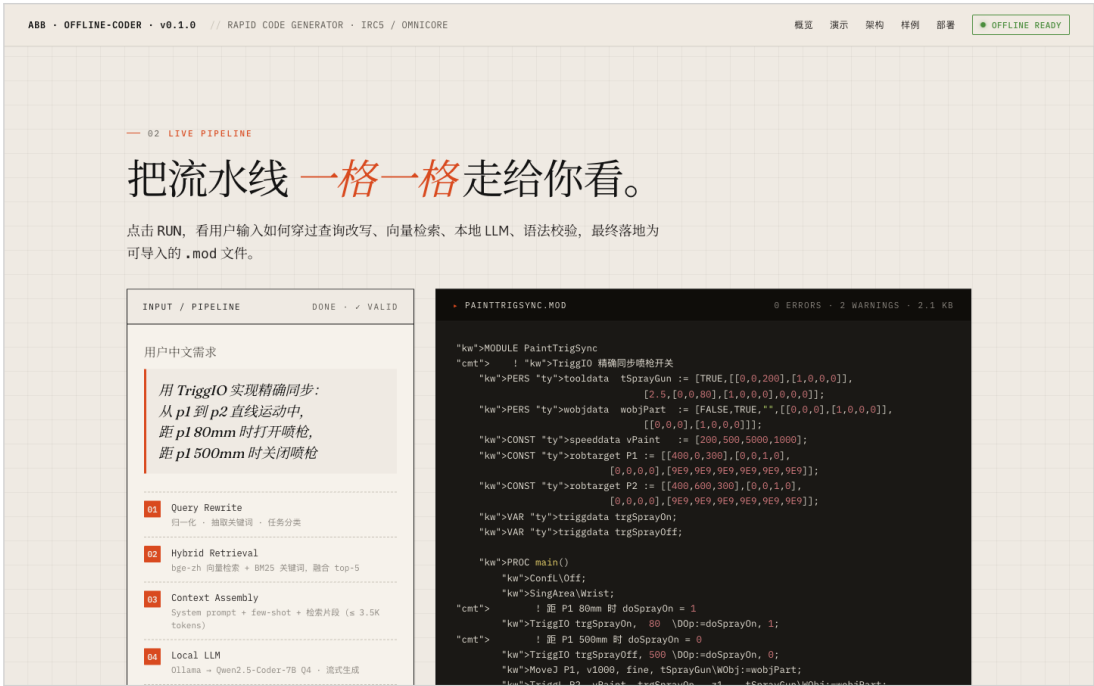


图 4. 在线演示页的交互式 Pipeline 演示（六步流水线动画 + 真实 .mod 流式输出）。

表 4. 关键技术选型

层	选型	理由
本地大模型	Ollama + Qwen2.5-Coder-7B-Instruct (Q4_K_M)	代码专精、4.5 GB 可 CPU 推理；3B 为弱机回退
嵌入模型	BAAI/bge-small-zh-v1.5（512 维, 92 MB）	中文语义强、CPU 单句 <50 ms
向量库	ChromaDB（内嵌持久化）	无独立服务进程，随项目落盘
关键词检索	rank-bm25（内存版 BM25Okapi）	精确指令名召回，启动即建索引
文档解析	PyMuPDF / BeautifulSoup / 自研 mod_parser	解析 ABB PDF 手册、HTML、历史.mod
CLI / 渲染	Typer + Rich	子命令清晰、终端语法高亮
配置 / 校验	Pydantic + pydantic-settings	环境变量覆盖、类型安全
语言 / 工具	Python 3.10+ ·pytest ·ruff ·mypy	工程化质量保障

5 关键技术实现

5.1 中文查询改写：低成本提升检索命中

用户口语化的中文（如“让喷枪沿直线扫过去”）与手册中的英文术语（MoveL straight path）存在语义鸿沟。项目实现了**基于词典 + 规则的轻量查询改写器**（不调用 LLM，毫秒级）：保留原始中文（利于向量召回）、追加官方英文术语（利于 BM25 召回），并识别 7 类任务类别（直线/Z 字/圆弧扫描、TCP 标定、笔刷工艺、IO 控制、通用）。其中对“Z 字 / 之字 / 弓字”等同义写法做**空格归一化**后统一映射，提升鲁棒性。

Listing 1. 查询改写：中文 → 检索友好查询（节选）

```
_CN_TO_EN = {
    "直线扫描": "linear scan MoveL straight path",
    "z字":      "zigzag scan back forth raster pattern",
    "校准":     "calibration TCP toolframe calibrate",
    "雾化":     "atomization atom pressure",
    "同步":     "TriggIO TriggL TriggData synchronized trigger distance",
    # ... 共 40+ 条术语映射
}

def rewrite(query: str) -> RewrittenQuery:
    normalized = re.sub(r"\s+", "", query).lower() # 归一化：去空格, "Z 字"=="z字"
    extras, keywords = [], set()
    for cn, en in _CN_TO_EN.items():
        if cn.lower() in normalized:
            extras.append(en); keywords.update(en.split())
    category = _classify_category(query) # 7 类任务识别
    rewritten = f"{query}\n[英文术语映射]: {'; '.join(extras)}\n[任务类别]: {category.value}"
    return RewrittenQuery(query, rewritten, category, tuple(sorted(keywords)))
```

5.2 向量 + BM25 混合检索

为兼顾**精确关键词**（“如何写 MoveL”——BM25 强）与**模糊意图**（“沿直线扫过去”——向量强），检索层采用混合策略：两路分别取 Top- k ，归一化后线性融合。设某片段的向量得分为 s_{vec} 、BM25 得分为 s_{bm25} ，经 min-max 归一化得 $\hat{s}$ ，最终融合得分为：

$$\text{score} = \alpha \cdot \hat{s}_{\text{vec}} + (1 - \alpha) \cdot \hat{s}_{\text{bm25}}, \quad \alpha = 0.5.$$

两路按 chunk_id 合并去重，匹配关键词集合并集保留，最终取 Top-5 进入上下文装配。分词上对 RAPID 关键词**整体保留**（MoveL 不可拆），中文按字符切分，兼顾专业指令名的精确召回。

Listing 2. 混合检索的分数融合（节选）

```
def _merge_and_rerank(self, vec, bm25, alpha):
    vec_norm = _normalize([r.score for r in vec]) # min-max 归一化到 [0,1]
    bm25_norm = _normalize([r.score for r in bm25])
```

```
score_map = {}
for r, ns in zip(vec, vec_norm):
    score_map[r.chunk.chunk_id] = (r.chunk, alpha * ns)
for r, ns in zip(bm25, bm25_norm):          # 同 id 累加另一路加权分
    prev = score_map.get(r.chunk.chunk_id, (r.chunk, 0.0))[1]
    score_map[r.chunk.chunk_id] = (r.chunk, prev + (1 - alpha) * ns)
return sorted_results(score_map)           # 按融合分降序
```

5.3 上下文装配：来源标注 + 预算控制

检索片段在进入提示词前需被装配：按相关度降序、跨源去重、给每段加来源标签（[ABB 官方手册]、[示例代码]、[喷涂工艺] 等），代码片段单独用 ``rapid 包裹以便 LLM 识别。为防止超出模型上下文窗口，按“汉字 1 字 $\approx$ 1.5 token、英文 4 字符/token”估算并控制总预算（默认 ≤ 3500 token），高分片段优先填入。

5.4 本地 LLM 推理与提示工程

推理通过 Ollama HTTP 客户端调用本地量化模型，内置连接重试（指数退避 2 次）与模型缺失自动降级（7B $\rightarrow$ 3B 回退）。提示词由三部分组成：

- **System Prompt**：约定“代码先于解释、永远输出完整 MODULE、大小写规范、每行分号、中文注释、安全模板（ConfL\Off; SingArea\Wrist;）、禁止虚构不存在的指令”等 10 条铁律；
- **Few-shot 示例**：6 个“需求 $\rightarrow$ 代码”范例（直线/Z 字/圆弧/TriggIO 同步/ TCP 校准/参数化扫描），让模型对齐输出风格；
- **检索上下文**：本次命中的官方手册片段，动态注入。

创新点·RAG 注入让小模型“查得到、不乱编”

实测中，未联网的本地模型借助 RAG 从 7497 个片段里检索到官方 MToolTCPCalib、TriggIO gunon, 0.2\Time\DOp:=gun 等 **专业指令的真实用法**并正确复用——这是裸模型（无检索）难以做到的，直接验证了“离线 RAG”路线的有效性。

5.5 RAPID 后处理：生成—校验闭环

LLM 输出不能直接上机。后处理依次完成**模块包裹** $\rightarrow$ **格式化** $\rightarrow$ **静态校验**。其中校验器是本项目的工程亮点：在不做完整 parser 的前提下（成本太高），用正则与栈式匹配实现 11 类**高频错误检查**，并赋予**可追溯错误码**，现场工程师可据码定位问题。

创新点·把“现场才会暴露的错误”提前到生成时拦截

表 5. RAPID 静态校验规则与错误码体系

错误码	级别	检查内容
MOD001	Error	缺少 MODULE 声明
BLK001/002/003	Error	块结构 (MODULE/PROC/IF/FOR/WHILE...) 未配对、错配、未闭合 (栈式匹配)
STR001	Error	字符串引号不配对
CASE001	Warning	关键字大小写不规范 (RAPID 大小写敏感)
SEM001	Warning	语句行末疑似缺少分号
<i>IRC5P 专项 (仅 controller=IRC5P 时启用)</i>		
PNT001	Error	PaintL/PaintC 参数不足, 疑似缺 brushdata 形参
TCP001	Error	tooldata 仍为默认占位 TCP [0,0,200] (strict-tcp 开启时)
IO001	Error	引用了不在控制器 EIO.cfg 白名单中的 IO 信号

IO001 白名单检查可避免上机后报 40212 信号未定义; TCP001 强制提示“默认 TCP 必须 4 点法重标定”; PNT001 用“已声明 brushdata 名称集合”与调用做交叉校验。这些都是把工业现场的高代价错误左移到生成即拦截的具体工程手段。

Listing 3. IO 白名单校验: 启发式识别 IO 信号并去重报错 (节选)

```

IO_SIGNAL_PREFIXES = ("do", "di", "ao", "ai", "go", "gi") # 业内 IO 命名约定

def _check_io_whitelist(lines, whitelist):
    allowed, reported, issues = set(whitelist), set(), []
    for ln, raw in enumerate(lines, 1):
        m = _IO_CALL_RE.match(_strip_comments(raw)) # SetDO/Reset/WaitDI...
        if not m: continue
        sig = m.group(2)
        if not sig.lower().startswith(IO_SIGNAL_PREFIXES): # 跳过 clock 等非 IO
            continue
        if sig in allowed or sig in reported: continue
        reported.add(sig) # 同一未知信号只报一次
        issues.append(ValidationIssue(ln, ERROR, "IO001",
            f"IO 信号 '{sig}' 不在 EIO.cfg 白名单中"))
    return issues

```

5.6 双控制器模式: IRC5 与 IRC5P 自适应

同一句需求, 针对不同控制器应生成不同风格的代码。项目把 controller 作为贯穿提示词、helpers、校验、Pack&Go 的一等参数, 实现一键切换。

Listing 4. 喷涂 helper: 同一调用按控制器产出不同 RAPID (Z 字扫描节选)

表 6. IRC5 与 IRC5P 输出风格对照

维度	IRC5（默认，通用控制器）	IRC5P（含 RobotWare Paint）
喷涂直线	MoveL + SetDO doSprayOn,1/0	PaintL target,speed,brushdata,zone,tool
喷涂圆弧	MoveC + SetDO	PaintC pMid,pEnd,...,brushdata,...
工艺数据	PERS num 占位参数	PERS brushdata 原生类型
开关枪时序	手动 SetDO 配 WaitTime	brushdata 的 preOpen/postClose 自动管理
新增校验	—	PNT001 / TCP001 / IO001

```
def zigzag_scan(origin, width, height, *, row_spacing=50.0, controller="IRC5", brush="bdMain", ...):
    is_paint = controller == "IRC5P"
    for i in range(num_rows):
        # 行起点高速定位（不喷）
        lines.append(f"MoveL {robtarg_inline(p_start)}, v500, fine, {tool}\\WObj:={wobj};")
        if is_paint:
            lines.append(f"PaintL {robtarg_inline(p_end)}, {speed}, {brush}, {zone}, {tool}\\WObj:={wobj};")
        else:
            lines += [f"SetDO {spray_signal}, 1;",
                      f"MoveL {robtarg_inline(p_end)}, {speed}, {zone}, {tool}\\WObj:={wobj};",
                      f"SetDO {spray_signal}, 0;"]
```

5.7 Pack&Go 一键交付

仅一个 .mod 并不能让控制器直接加载运行。项目的 --bundle 模式把单模块扩展为完整可加载目录，并处理 ABB 控制器特有的编码细节：.sys / .pgf 用 ISO-8859-1 + CRLF 写入（老 RobotWare 默认按 Latin-1 解码，中文会导致加载失败），BASE.sys 强制纯 ASCII 并把中文说明剥离到同目录 README；写入采用先写文件、最后落 MANIFEST.ok 的策略，使“半写状态”可被外部识别。

output/MyPaintLine/ —可直接加载控制器

PaintProgram.mod
主程序 (PaintL+brushdata)

BASE.sys
Home 位 + 错误恢复 trap

T_ROB1.pgf
XML 任务加载清单

README.md
3 种加载 + 上线 4 步

5.8 一键离线迁移

为把整套环境搬到隔离工业 PC，项目提供 bundle.sh / restore.sh 零参数脚本：自动收集源码 + ABB PDF 手册 + ChromaDB 向量库 + bge 嵌入模型 + Ollama LLM + 离线 Python wheels，打成单个 tar.gz 并附 SHA256 校验；新设备解压后一

行 `restore.sh` 完成“检查 Python → 建虚拟环境离线装依赖 → 合并 Ollama 模型 → 验证向量库 → 自检”五步。实测精简离线包约 192 MB。

6 创新点总结

1. **完全离线的工业领域代码生成**：在无网工业 PC（CPU、无 GPU）上跑通“自然语言 → 可上机程序”，数据全程不出厂，破解云端工具无法落地的行业痛点。
2. **领域专精 RAG 知识注入**：以 10 本真实 ABB 官方手册构建 7497 片段向量库，向量 + BM25 混合检索，使小模型也能正确引用专业指令、显著抑制幻觉。
3. **IRC5 / IRC5P 双控制器自适应**：`controller` 作为贯穿提示词、代码生成、校验、交付的一等参数，一键切换 `MoveL+SetDO` 与 `PaintL/PaintC+brushdata` 两套工艺风格。
4. **生成—校验闭环（11 类规则 + 错误码）**：把 IO 未定义、默认 TCP、笔刷缺失等现场高代价错误左移到生成时拦截。
5. **Pack&Go 一键交付**：从一句话需求直接产出 `.mod+BASE.sys+T_ROB1.pgf+README` 可加载目录，并正确处理控制器编码。
6. **工程化离线迁移**：一键 `bundle/restore` 把模型、向量库、手册整体搬运、校验、自检，真正可在隔离设备复现。
7. **安全护栏内建**：TCP 占位检测、IO 白名单、关键字段人工复核提示，贯彻“AI 生成、人工把关”的工业安全理念。

7 工程质量与实测

7.1 代码规模与测试

测试方面，项目建立了 22 个测试文件、约 149 个 `test_*` 用例函数的单元测试套件（README 徽章记录 124 项通过；随 IRC5P / Pack&Go 能力扩展，用例持续增长），覆盖查询改写、混合检索模型、RAPID 校验（含 IRC5P 专项）、模块模板、喷涂 helpers、Pack&Go 系统包、编码安全、IO 白名单解析等。核心纯逻辑模块覆盖率高（如 `config.py`、`query_rewriter.py`、`painting_helpers.py` 达 100%）。数据模型采用 `frozen dataclass` 保证不可变，副作用集中在 Ollama / ChromaDB 边界，利于隔离测试。

7.2 知识库与模型规格

7.3 性能基准（工业 PC：i5 + 16 GB RAM，无 GPU）

表 7. 代码规模与模块分布

模块	行数	职责	说明
rapid/	1448	校验/模板/喷涂 helper/Pack&Go	后处理与安全护栏核心
knowledge/	914	PDF/MOD/HTML 解析 + 切片	离线知识库构建
rag/	776	改写/嵌入/向量库/混合检索/装配	检索增强主链路
cli/	608	gen/chat/kb/init/doctor	用户交互层
llm/	354	Ollama 客户端 + Prompt 模板	本地推理
合计	≈4500	37 个源文件	全部 Python 3.10+

表 8. 离线知识库与模型规格

项	值
官方手册	10 本真实 ABB 手册，约 54MB（RAPID 参考手册、Painting PowerPac、IRB 52/5400/5710 等）
向量库	ChromaDB 7497 个切片 / 约 52MB
嵌入模型	BAAI/bge-small-zh-v1.5, 512 维，约 92MB，CPU 推理
主 LLM	Qwen2.5-Coder-7B-Instruct Q4_K_M（约 4.5 GB）
回退 LLM	Qwen2.5-Coder-3B-Instruct Q4_K_M（弱配工业 PC）
离线包	精简版约 192MB（含源码 + 手册 + 向量库 + 嵌入模型）

表 9. 端到端性能实测

操作	预期时间
知识库检索（Top-5）	< 0.5 s
LLM 推理（Qwen 7B Q4）	15–30 s
后处理 + 校验	< 0.2 s
端到端单次生成	20–35 s

8 应用案例：车门外板 Z 字喷涂

以需求“在 600×400 平板上做 Z 字扫描喷涂，行距 50mm ”为例，同一句话在两种控制器模式下生成不同但均通过校验的程序。下图为项目演示页展示的若干真实生成 .mod 样本（直线扫描、TriggIO 同步、TCP 校准）。



图 5. 演示页展示的真实生成 RAPID 样本（含体积与校验耗时铭牌）。

8.1 IRC5 模式输出（MoveL + SetDO）

Listing 5. DoorPaint_IRC5.mod（节选）——通用控制器，手动 SetDO 开关枪

```
MODULE DoorPaint_IRC5
! 600x400 平板 Z 字扫描喷涂 (IRC5 模式样例)
PERS tooldata tSprayGun := [TRUE, [[0,0,200],[1,0,0,0]], [2.5,[0,0,80],[1,0,0,0],0,0,0]];
PERS wobjdata wobjPart := [FALSE, TRUE, "", [[0,0,0],[1,0,0,0]], [[0,0,0],[1,0,0,0]]];
CONST speeddata vPaint := [200, 500, 5000, 1000];
PROC main()
  Confl\Off;
  SingArea\Wrist;
  ! Z 字扫描喷涂
  MoveL [[0,0,300],[1,0,0,0],[0,0,0,0],[9E9,9E9,9E9,9E9,9E9,9E9]], v500, fine, tSprayGun\WObj:=
wobjPart;
  SetDO doSprayOn, 1;
  MoveL [[600,0,300],[1,0,0,0],[0,0,0,0],[9E9,9E9,9E9,9E9,9E9,9E9]], vPaint, z10, tSprayGun\WObj:=
wobjPart;
  SetDO doSprayOn, 0;
  ! ... 逐行 Y 偏移 50mm 往返扫描 ...
ENDPROC
```

ENDMODULE

8.2 IRC5P 模式输出 (PaintL + brushdata)

Listing 6. DoorPaint_IRC5P.mod (节选) ——Paint 控制器, brushdata 自动管理喷涂时序

```
MODULE DoorPaint_IRC5P
  ! 600x400 平板 Z 字扫描喷涂 (IRC5P 模式样例)
  PERS tooldata tSprayGun := [TRUE, [[0,0,200],[1,0,0,0]], [2.5,[0,0,80],[1,0,0,0],0,0,0]];
  PERS wobjdata wobjPart := [FALSE, TRUE, "", [[0,0,0],[1,0,0,0]], [[0,0,0],[1,0,0,0]]];
  CONST speeddata vPaint := [200, 500, 5000, 1000];
  ! [WARNING] brushdata 字段顺序随 RobotWare Paint 版本不同, 上控制器前须对照 Brush Table 校核!
  PERS brushdata bdMain := [80, 300, 50, 50, 50, 0, FALSE, "bdMain", 0];
  PROC main()
    ConfL\Off;
    SingArea\Wrist;
    ! Z 字扫描喷涂 (IRC5P PaintL)
    MoveL [[0,0,300],[1,0,0,0],[0,0,0,0],[9E9,9E9,9E9,9E9,9E9,9E9]], v500, fine, tSprayGun\WObj:=
wobjPart;
    PaintL [[600,0,300],[1,0,0,0],[0,0,0,0],[9E9,9E9,9E9,9E9,9E9,9E9]], vPaint, bdMain, z10,
tSprayGun\WObj:=wobjPart;
    ! ... 后续行 PaintL 走 brushdata 工艺 ...
  ENDPROC
ENDMODULE
```

两份程序均自动补全 tooldata/wobjdata/speeddata 声明、安全模板与中文注释, 并通过静态校验。配合 --bundle 即生成 BASE.sys 与 T_ROB1.pgf, 形成可直接拖入 RobotStudio 加载的完整目录。

9 安全、合规与伦理

- **数据不出厂**: 全离线运行, 需求/工艺/历史程序不上传任何外部服务, 符合工业信息安全要求;
- **人在回路**: 生成代码**必须**在 RobotStudio 仿真验证后方可上机; 速度、转弯区、TCP 等安全字段强制人工复核;
- **安全护栏**: 默认 TCP 占位检测 (TCP001)、IO 白名单 (IO001)、BASE.sys 错误恢复 trap 与 Home 位回退;
- **知识产权合规**: 手册由用户在有网环境**手动合法获取**后入库, 项目本身不抓取、不分发受版权保护的 ABB 资料;
- **商标声明**: 本项目与 ABB Ltd. 无任何关联, 亦不被其支持或认可; “ABB” “RobotStudio” “RAPID” 均为 ABB Ltd. 商标, 本项目为独立第三方实用工具, 以 MIT 许可证开源。

10 局限性与未来工作

10.1 当前局限

- 小模型对生僻 RAPID 指令理解仍不完美，建议先用 `kb inspect` 检查检索质量；
- 校验为轻量正则/栈式匹配，非完整语法分析，深层语义错误仍需仿真兜底；
- `brushdata` 字段顺序随 RobotWare Paint 版本而异，需现场对照 Brush Table 校核。

10.2 演进路线

- ✓ v0.1: CLI + RAG + 喷涂 few-shot（已完成）
- ✓ v0.2: IRC5P 模式 + PaintL/PaintC + brushdata + Pack&Go（已完成）
- v0.3: 扩充 IRB 5500 / 6700 等更多机型专门 few-shot
- v0.4: 可选 GUI（Textual TUI 或本地 Web）
- v0.5: 完整 RAPID parser，提升校验深度至语义层
- v0.6: RobotStudio Add-In（C# 桥接），一键回灌仿真

10.3 方法论可迁移性

本项目沉淀的“**离线 RAG + 领域校验闭环 + 一键交付**”范式不局限于 ABB：更换知识库、提示词与校验规则后，可平移至 KUKA KRL、FANUC TP、安川 INFORM 等其它封闭工业机器人语言，乃至 PLC（结构化文本 ST）等工控代码生成场景，具备清晰的产品化与推广潜力。

11 结论

abb-offline-coder 针对喷涂机器人编程“高门槛 + 无网隔离”的真实困境，提出并完整实现了一套**完全离线、领域专精、生成即校验**的中文 AI 编程助手。通过本地量化大模型与官方手册 RAG 的结合、IRC5/IRC5P 双控制器自适应、11 类静态校验闭环与 Pack&Go 一键交付，系统把“写一段合格喷涂程序”从专家级的数小时工作，降为数十秒的自然语言交互加人工复核，且全程数据不出厂。项目以约 4500 行 Python、149 个测试用例、7497 片段知识库与 20-35 秒端到端性能，证明了“无网工业现场可用的 AI 编程助手”这一路线的技术可行性与工程成熟度，并为更广泛的封闭工业生态代码生成提供了可复制的方法论范式。

参考资料与项目信息

- 项目源码: <https://github.com/Hollis36/abb-offline-coder> (MIT 许可证)
- 在线演示页: <https://hollis36.github.io/abb-offline-coder/>
- ABB *RAPID Reference Manual—Instructions, Functions and Data types*(文档号 3HAC16581)
- ABB *Painting PowerPac Operating Manual* (3HNA019758)、*Application Manual—Robot-Ware Paint*
- ABB *Product Manual*: IRB 52 / 5400 / 5710 系列
- BAAI bge-small-zh-v1.5 中文嵌入模型; Qwen2.5-Coder 开源代码大模型; Ollama 本地推理框架
- ChromaDB 向量数据库; rank-bm25; PyMuPDF; Typer / Rich / Pydantic

项目演示页提供的引用格式 (@software):

```
@software{abb_offline_coder,
  title = {abb-offline-coder: Offline AI assistant for ABB RAPID painting robots},
  author = {Hollis36},
  year = {2026},
  url = {https://github.com/Hollis36/abb-offline-coder},
  note = {Demo: https://hollis36.github.io/abb-offline-coder/, MIT License}
}
```